TRACE

Technical Handbook

Version 2.0 • April 2026

Hybrid LLM + Rule Engine + XGBoost Cascade

for Automotive Warranty Claim Adjudication

TRACE — Technical Handbook

Version: 2.0

Date: April 2026

Dataset: synthetic_warranty_claims_v9.csv (100,000 rows, 2019–2025)

Preface

This handbook documents the TRACE (Technical Resolution and Claims Evaluation) system — a hybrid LLM + Rule Engine + XGBoost pipeline for automotive warranty claim adjudication. It is written so that a new team member, external auditor, or future maintainer can understand what the system does, why each piece exists, and how everything connects, without needing to read the source code.

Who this is for: Software engineers onboarding to the project, ML engineers evaluating model behavior, QA engineers writing tests, and product stakeholders who want to understand system decisions.

How to use it: Read Chapter 1 for the big picture, Chapter 2 for architecture, then jump to whichever chapter is relevant. The file-by-file reference (Chapter 4) is the backbone — every runtime module is documented there with key contents, data flow, and design notes.

---

1. Project Overview

1.1 Problem Statement

Automotive warranty claims arrive daily at tier-1 suppliers and OEMs. Each claim contains a diagnostic trouble code (DTC), technician observations, and a voltage reading. A human analyst must decide: Is this a **Production Failure** (warranted, supplier's responsibility), a **Customer Failure** (not warranted, owner's responsibility), or **According to Specification** (no fault found, NTF)?

This process is slow, inconsistent, and expensive. TRACE automates it by combining three decision layers:

1. **Deterministic rules** — catch known patterns (over-voltage, moisture keywords, DTC prefix categories) with high confidence.
2. **XGBoost classifiers** — generalize to patterns the rules don't cover, producing a probability-weighted failure analysis and warranty decision.
3. **LLM semantic understanding** — when available, categorizes free-text notes into structured data and polishes the output explanation.

1.2 Solution Approach

TRACE uses a **six-stage prediction pipeline** that combines all three layers:

Stage	Component	Purpose
1	LLM Understanding	Semi-semantic categorization of technician notes
2	Rule Engine	Deterministic pattern matching for known fault scenarios
3	Feature Translation	Convert raw inputs into ML-ready features (LLM or fallback)
4	XGBoost Scoring	Cascaded classifiers: Failure Analysis → Warranty Decision
5	Score Combination	Weighted blend of rule confidence, ML confidence, and LLM signal
6	Output Formatting	Natural-language reasoning (LLM or fallback)

The pipeline degrades gracefully: if no API key is set, stages 1, 3, and 6 fall back to deterministic local code, and the system still produces results via "Rule+ML" or "ML" engine tags.

1.3 Inputs and Outputs

Input (ClaimRequest):

Field	Type	Description	Example
fault_code	string	DTC code(s), comma-separated	"P0562" or "U0100, C0045"
technician_notes	string	Free-text observations	"Engine overheating, low idle"
voltage	float	Measured battery/charging voltage	14.2

Output (ClaimResponse):

Field	Type	Values	Description
status	string	Approved, Rejected, Needs Manual Review	Top-level disposition
failure_analysis	string	—	Root cause prediction (e.g. "Sensor short due to moisture")
warranty_decision	string	Production Failure, Customer Failure, According to Specification	Warranty liability assignment
confidence	float	0–100	Decision confidence

Field	Type	Values	Description
`reason`	string	—	Human-readable explanation
`matched_complaint`	string	—	`How technician notes were interpreted`
`decision_engine`	string	LLM+Rule+ML, Rule+ML, LLM+ML, ML	Which engines contributed

1.4 Stakeholders and Use Cases

- **Warranty analysts** — submit claims, receive automated recommendations
- **Engineering QA** — review "Needs Manual Review" escalations
- **Product managers** — track warranty claim trends and approval rates

1.5 Key Design Goals

1. Deterministic override — rules must be able to firmly reject or approve high-confidence patterns regardless of ML output.

2. Graceful degradation — the system produces a result even when the LLM is unavailable.

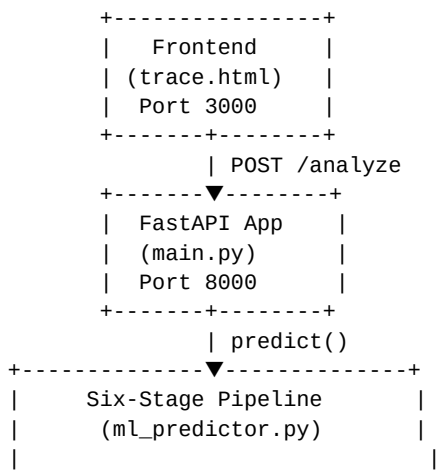
3. Explainability — every output includes a `reason` string and a `decision_engine` tag showing which layers contributed.

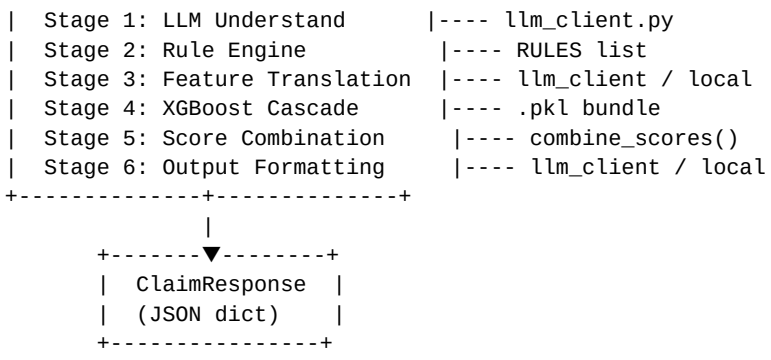
4. Cascade architecture — the WD classifier receives FA probabilities as features, so the root-cause prediction informs the warranty decision.

5. No data leakage — transformers are fit on the training split only; test-set statistics never leak into the model.

2. Architecture Overview

2.1 System Architecture Diagram





2.2 Component Descriptions

Component	File	Role
API layer	`backend/main.py`	FastAPI endpoints, request/response schemas, OCR scanning
Prediction engine	`backend/ml_predictor.py`	Six-stage pipeline: rules, feature extraction, XGBoost, scoring
LLM integration	`backend/llm_client.py`	OpenAI/OpenRouter provider abstraction, retry logic, JSON parsing
Logging	`backend/logging_config.py`	Centralized logging format, DecisionLogger helper
Model evaluation	`backend/evaluate_model.py`	Metrics computation, cross-validation, cascade calibration check
Frontend	`frontend/trace.html`	Single-page UI with DTC input, voltage, notes, and result display
Container orchestration	`docker-compose.yml`	Backend + frontend containers with health checks

2.3 How Components Interact

1. **User** fills out the claim form on `trace.html` and clicks "Analyze Claim".
2. **Frontend** sends `POST /analyze` with `{fault_code, technician_notes, voltage}`.
3. **FastAPI** (`main.py`) receives the `ClaimRequest`, calls `ml_predictor.predict()`.
4. **predict()** orchestrates the six-stage pipeline:

- If `OPENROUTER_API_KEY` or `OPENAI_API_KEY` is set and notes > 5 chars, calls `llm_client.understand_claim_with_retry()`.
- Runs `run_rules()` — deterministic pattern matching.
- Prepares features for ML (via LLM translation or local fallback).
- Runs `run_ml()` — XGBoost cascade scoring.
- Calls `combine_scores()` — weighted blend with LLM signal.

- Calls `format_output()` (LLM) or `assemble_output_from_fields()` (fallback).

5. Result is returned as `ClaimResponse` JSON and rendered in the frontend.

2.4 Technology Stack Choices

Choice	What	Why
FastAPI	API framework	`Async-capable, automatic OpenAPI docs, Pydantic validation`
XGBoost	ML classifier	`Superior accuracy over RandomForest for this dataset (per evaluation benchmarks); handles sparse features well`
scikit-learn	Preprocessing	`Industry-standard TF-IDF, OHE, StandardScaler, LabelEncoder`
OpenRouter	LLM provider	`Free-tier API access; supports multiple models; arcee-ai/trinity-large-preview:free for cost efficiency`
`OpenAI (gpt-4o-mini)`	LLM fallback	`Higher quality responses; used when OPENAI_API_KEY is set`
EasyOCR	Image scanning	`Extract DTC codes and voltage from technician photos`
Docker + nginx	Deployment	`Lightweight containerized deployment; nginx serves static frontend`
`Vanilla HTML/CSS/JS`	Frontend	`No framework overhead; rapid iteration; single trace.html file`

2.5 Key External Dependencies

Dependency	Version	Purpose
fastapi	`0.111.0`	REST API framework
uvicorn	`0.30.1`	ASGI server
scikit-learn	`1.5.0`	Preprocessing transformers, metrics
xgboost	`≥2.0.0`	Gradient boosting classifiers
scipy	`1.13.1`	`Sparse matrix operations (hstack)`
numpy	`1.26.4`	Array operations
pandas	`2.2.2`	DataFrame operations
openai	`≥1.0.0`	OpenAI Python client

Dependency	Version	Purpose
requests	≥2.31.0	HTTP client for OpenRouter
python-dotenv	≥1.0.0	Environment variable loading
easyocr	latest	OCR for image scanning
pillow	latest	Image processing
torch	latest	Backend for EasyOCR

3. Project Structure

3.1 Annotated Directory Tree

```
capProj-2/
+-- AGENTS.md                          # Opencode agent configuration and project docs
+-- README.md                           # Quick-start guide
+-- docker-compose.yml                  # Full-stack container orchestration
|
+-- backend/                            # FastAPI application
|   +-- main.py                         # API endpoints, Pydantic schemas, OCR scanning
|   +-- ml_predictor.py                 # Core prediction engine (941 lines)
|   +-- llm_client.py                  # LLM provider abstraction (476 lines)
|   +-- logging_config.py               # Centralized logging (61 lines)
|   +-- evaluate_model.py              # Model evaluation script (528 lines)
|   +-- resave_bundle.py                # Utility: convert .pkl &rarr; .joblib (9 lines)
|   +-- test_api_timing.py             # LLM API timing benchmark (124 lines)
|   +-- ml_predictor_DecisionTree.py   # Archived alternative model (uses DecisionTree)
|   +-- requirements.txt               # Python dependencies
|   +-- .env                           # API keys (not in repo)
|   +-- .env.example                   # Environment template
|   +-- synthetic_warranty_claims_v9.csv  # Training dataset (100K rows)
|   +-- trace_models.pkl              # Trained model bundle (generated)
|   +-- Dockerfile                     # Backend container
|   +-- backup/                        # Previous version backups
|   +-- dataset_gen/                   # Synthetic dataset generation scripts
|   |   +-- generate_dataset_v3.py
|   |   +-- generate_dataset_v4.py
|   |   +-- generate_dataset_v5.py
|   |   +-- generate_dataset_v6.py
|   |   +-- generate_dataset_v9(1).py
|   +-- tests/                         # Test suite
|       +-- test_ml_predictor.py        # Core unit and integration tests
|       +-- test_llm_client.py         # LLM client unit tests
|       +-- test_llm_client_logging.py # LLM logging tests
|       +-- test_logging_config.py     # Logging configuration tests
|       +-- test_main_logging.py       # API logging tests
|       +-- test_ml_predictor_logging.py # ML logging tests
|       +-- test_e2e.py                # End-to-end pipeline tests
|       +-- test_predictor_llm.py      # Predictor + LLM integration
|       +-- test_openai_integration.py # OpenAI API integration tests
|       +-- test_resilience.py         # Failure scenario resilience tests
```

```
|
+-- frontend/                          # Static frontend
|   +-- trace.html                     # Production frontend (served by nginx)
|   +-- index.html                     # Development frontend (same UI, points to localhost:8000)
|   +-- nginx.conf                     # Nginx configuration for SPA
|   +-- Dockerfile                     # Frontend container (nginx:1.27-alpine)
|
+-- docs/                              # Documentation
|   +-- model-improvement-findings-2026-04-19.md
|   +-- plans/                         # Architecture and improvement plans
|       +-- xgboost-implementation-plan.md
|
+-- thoughts/                          # Opencode research and decision logs
    +-- shared/
        +-- plans/                     # Implementation plans
        +-- research/                  # Research notes
        +-- reviews/                   # Code reviews
```

3.2 File Naming Conventions

Pattern	Convention	Example
Python modules	`snake_case`	`ml_predictor.py, llm_client.py`
Test files	`test_<module>.py`	`test_ml_predictor.py, test_e2e.py`
Dataset files	`synthetic_warranty_claims_vN.csv`	`synthetic_warranty_claims_v9.csv`
Model bundles	`trace_models.pkl`	`Generated by train_and_save()`
Config files	lowercase, dot-prefix	`.env, nginx.conf`
Docker files	`Dockerfile`	One per container

---

4. File-by-File Reference

4.1 `backend/main.py`

Role: FastAPI application serving the `/analyze` and `/scan-image-easyocr` endpoints.

Why it exists: The API layer decouples the frontend from the prediction engine. It handles HTTP concerns (CORS, request validation, error handling) so `ml_predictor.py` stays focused on prediction logic.

Key Contents

Element	Type	Purpose
`ClaimRequest`	Pydantic model	`Input schema: fault_code, technician_notes, voltage`

Element	Type	Purpose
`ClaimResponse`	Pydantic model	`Output schema: status, failure_analysis, warranty_decision, confidence, reason, matched_complaint, decision_engine`
`extract_dtc_codes()`	function	`Regex-based DTC extraction from OCR text`
`extract_voltage()`	function	`Regex-based voltage extraction from OCR text`
`/scan-image-easyocr`	endpoint	`Accepts uploaded image, runs EasyOCR, returns extracted DTC codes and voltage`
`/analyze`	endpoint	`Core prediction: accepts ClaimRequest, calls ml_predictor.predict(), returns ClaimResponse`
`/`	endpoint	`Health check returning version and status`

How It Works

1. On startup, `load_dotenv()` loads API keys from `.env`.

2. The `ml_predictor` module auto-loads the `.pkl` model bundle on first `predict()` call (lazy initialization via `_bundle` global).

3. `POST /analyze` validates the request via Pydantic, calls `predict()`, and wraps the result in `ClaimResponse`.

4. `POST /scan-image-easyocr` runs EasyOCR on an uploaded image, extracts DTC codes and voltage via regex, returns structured data for the frontend to pre-fill.

Dependencies

- **Depends on:** `ml_predictor.predict`, `logging_config.get_logger`, `easyocr`, `PIL`, `dotenv`
- **Consumed by:** Frontend `trace.html` (via HTTP)

Design Notes

- CORS is wide open (`allow_origins=["*"]`) — intended for development. Restrict in production.
- The OCR endpoint initializes `easyocr.Reader(['en'])` at module-level, which takes ~5 seconds on first import. This is acceptable for a long-running server but would be problematic in serverless environments.
- Error handling returns 500 with a generic message; the traceback goes to logs but not the client.

4.2 `backend/ml_predictor.py`

Role: Core prediction engine implementing the six-stage LLM + Rule + ML pipeline.

Why it exists: This is the heart of TRACE. It encapsulates the entire decision pipeline: rule engine, feature extraction, XGBoost cascade inference, and score combination. All prediction logic lives here so it can be tested independently from the API layer.

Key Contents

Element	Type	Purpose
`RULES`	list of 9 dicts	Deterministic automotive rules with match lambdas, outcomes, and confidences
`HIGH_VALUE_DTCS`	list of ~90 strings	DTC codes flagged as one-hot features for the ML model
`KNOWN_COMPLAINTS`	list of 14 strings	Canonical complaint labels for fuzzy matching
`CONFIDENCE_THRESHOLD_FIRM`	constant	85.0 — above this, decision is firm
`CONFIDENCE_THRESHOLD_MANUAL`	constant	65.0 — below this, status is "Needs Manual Review"
`RULE_WEIGHT_AGREE`	constant	0.70 — rule weight when rule and ML agree
`ML_WEIGHT_AGREE`	constant	0.30 — ML weight when rule and ML agree
`LLM_WEIGHT`	constant	0.15 — LLM signal weight when present
`extract_dtc_features()`	function	Parses DTC string → dict with prefix flags, count, high-value one-hots
`match_complaint()`	function	Fuzzy-maps free-text notes → KNOWN_COMPLAINTS label
`train_and_save()`	function	Trains XGBoost cascade, saves bundle to `trace_models.pkl`
`load_models()`	function	Loads `trace_models.pkl` or trains if missing
`run_rules()`	function	Evaluates 9 rules in priority order; first match wins
`run_ml()`	function	Runs cascaded XGBoost inference on feature dict
`combine_scores()`	function	Weighted blend of rule, ML, and LLM signals; threshold logic
`assemble_output_from_fields()`	function	Fallback output formatter when LLM is unavailable
`predict()`	function	Orchestrates all 6 stages; returns final ClaimResponse dict

How It Works

`predict()` (lines 836–925) is the single entry point. It:

1. Lazy-loads the model bundle if `_bundle is None`.
2. Checks for LLM availability (`OPENROUTER_API_KEY` set and notes > 5 chars).
3. **Stage 1**: Calls `llm_client.understand_claim_with_retry()` — gets category, severity, failure analysis.
4. **Stage 2**: Calls `run_rules()` — evaluates all 9 rules; first match wins, returns `{"rule_fired": False}` if none match.
5. **Stage 3**: If LLM is available, calls `llm_client.translate_to_ml_features()`. Otherwise, uses `extract_dtc_features() + match_complaint()` locally. Default values fill in: supplier="Unknown", mileage_km=50000.0, year=2024, claim_age=1, voltage=14.2 (if not provided).
6. **Stage 4**: Calls `run_ml(features)` — builds sparse feature matrix, runs cascaded XGBoost (FA then WD).
7. **Stage 5**: Calls `combine_scores(rule_result, ml_result, llm_stage1)` — produces final status, confidence, and decision_engine tag.
8. **Stage 6**: If LLM is available, calls `llm_client.format_output()`. Otherwise, calls `assemble_output_from_fields()`.

`train_and_save()` (lines 278–451) trains the model:

1. Loads `synthetic_warranty_claims_v9.csv`.
2. Fills NAs in DTC, Customer Complaint, Failure Analysis, Warranty Decision.
3. Fits `LabelEncoder` on full dataset (safe for targets — no leakage).
4. Splits 80/20 with `random_state=42`. Split happens **before** any `fit_transform` call.
5. Engineers post-split features: `mileage_bracket`, `claim_age`, `voltage_bracket`, `dtc_count_bracket`, interaction features.
6. Fits 13 transformers on training data only: OHE (complaint, supplier, mileage bracket, voltage bracket, DTC count bracket), TF-IDF (DTC text, max_features=40), StandardScaler (mileage, year, claim_age, voltage).
7. Builds sparse feature matrix via `scipy.sparse.hstack`.
8. Trains XGBoost FA classifier (1000 estimators, max_depth=10, lr=0.02).
9. Generates OOF FA probabilities via 5-fold `cross_val_predict` for WD cascade input.
10. Trains XGBoost WD classifier on augmented matrix (original features + FA probability vector).
11. Saves bundle `dict` to `trace_models.pkl` via pickle.

`run_rules()` (lines 465–492) evaluates rules in order:

Priority	Rule ID	Condition	Status	WD	Confidence
1	`over_voltage`	`voltage > 16.0`	Rejected	Customer Failure	`93.0%`
2	`low_voltage`	`voltage < 11.0`	Rejected	Customer Failure	`95.0%`
3	moisture	keyword match in notes	Rejected	Customer Failure	`91.0%`
4	`physical_damage`	keyword match in notes	Rejected	Customer Failure	`88.5%`
5	ntf	keyword match in notes	Approved	According to Spec	`95.0%`
6	`u_code`	U-series DTC	Approved	Production Failure	`57.0%`

Priority	Rule ID	Condition	Status	WD	Confidence
7	`p_code_engine`	P0-series + symptom keywords	Approved	Production Failure	`80.5%`
8	`c_code`	C-series DTC	Approved	Production Failure	`80.0%`
9	`b_code`	B-series DTC	Approved	Production Failure	`80.0%`

`combine_scores()` (lines 664–808) performs the weighted blend:

- **Agreement (rule + ML agree):** `0.70 x rule_conf + 0.30 x ml_conf + 5.0 bonus`. If LLM also agrees, weights shift to include `LLM_WEIGHT (0.15)`.
- **Disagreement (rule + ML disagree):** `0.55 x rule_conf + 0.35 x ml_conf`. If LLM agrees with one side, it tilts the blend.
- **No rule fired:** ML confidence alone, multiplied by `(1 - LLM_WEIGHT) + LLM_WEIGHT x llm_scaled`. If LLM confidence < 0.3, a weak-input penalty of 0.85 is applied.
- **Status determination:** ≥85% → firm decision (Approved/Rejected from rule or ML), 65–85% → rule/ML status, <65% → "Needs Manual Review".
- **decision_engine tag:** "LLM+Rule+ML" (all three), "Rule+ML" (rule + ML, no LLM signal), "LLM+ML" (no rule, LLM present), "ML" (neither rule nor LLM).

Dependencies

- **Depends on:** `llm_client` (for Stages 1, 3, 6), `logging_config`, `sklearn`, `xgboost`, `scipy.sparse`, `pandas`, `numpy`, `pickle`
- **Consumed by:** `main.py` (via `predict()`), `evaluate_model.py`, test suite

Design Notes

- The module-level `_bundle` global caches the loaded model to avoid re-loading on each request.
- Default feature values (supplier="Unknown", mileage=50000, year=2024, claim_age=1, voltage=14.2) are used when the LLM feature-translation path fails. These defaults are reasonable mid-range values for automotive warranty claims.
- The `CONFIDENCE_THRESHOLD_FIRM = 85.0` and `CONFIDENCE_THRESHOLD_MANUAL = 65.0` constants are deliberately chosen to create three tiers: firm decisions (≥85%), cautious decisions (65–85%), and mandatory human review (<65%).
- Rule priority is intentional: voltage rules (over/under) are first because they are the most objective and reliable indicators. Keyword rules (moisture, physical damage, NTF) come next. DTC prefix rules come last because they have lower confidence and are more ambiguous.
- The `voltage_bracket` and `dtc_count_bracket` functions are duplicated between `train_and_save()` and `run_ml()`. This is intentional — they must stay in sync, and refactoring to a shared function would require passing the transformers bundle during training.

4.3 `backend/llm_client.py`

Role: LLM provider abstraction supporting OpenAI (gpt-4o-mini) and OpenRouter, with retry logic and JSON response parsing.

Why it exists: TRACE needs structured output from LLMs for three purposes: claim understanding (Stage 1), feature translation (Stage 3), and output formatting (Stage 6). This module provides a unified interface that handles provider selection, API calls, retry logic, and JSON parsing — keeping LLM concerns isolated from the prediction pipeline.

Key Contents

Element	Type	Purpose
`OPENAI_MODEL`	constant	"gpt-4o-mini" — OpenAI model to use
`OPENROUTER_MODEL`	constant	`"arcee-ai/trinity-large-preview:free" — OpenRouter model`
`_get_provider()`	function	`Returns "openai" or "openrouter" based on env vars`
`get_api_key()`	function	`Retrieves the appropriate API key from env vars`
`_call_llm()`	function	`Routes to OpenAI or OpenRouter based on provider`
`_call_openai()`	function	`Calls OpenAI Chat Completions API with JSON mode`
`_call_openrouter()`	function	Calls OpenRouter API via HTTP POST
`_parse_json_response()`	function	`Parses LLM JSON output with fallback defaults`
`understand_claim()`	function	`Stage 1: categorizes notes into one of 7 categories`
`understand_claim_with_retry()`	function	`Wraps understand_claim() with exponential backoff retry (2 attempts)`
`translate_to_ml_features()`	function	`Stage 3: extracts structured ML features from free-text`
`format_output()`	function	`Stage 6: produces polished natural-language output`
`CATEGORIZATION_PROMPT`	constant	`Prompt template for claim categorization (7 categories)`
`UNDERSTAND_CLAIM_PROMPT`	constant	`Prompt template for Stage 1 understanding`
`TRANSLATE_ML_FEATURES_PROMPT`	constant	`Prompt template for Stage 3 feature extraction`
`FORMAT_OUTPUT_PROMPT`	constant	`Prompt template for Stage 6 output formatting`

How It Works

Provider selection logic:

1. If `OPENAI_API_KEY` is set → use OpenAI (gpt-4o-mini).
2. Else if `OPENROUTER_API_KEY` is set → use OpenRouter (arcee-ai/trinity-large-preview:free).
3. Else → no LLM available; all LLM-dependent stages fall back to local code.

`understand_claim()` (lines 346–377): Sends a prompt asking the LLM to categorize the claim into one of 7 categories: `moisture_damage`, `physical_damage`, `ntf`, `electrical_issue`, `engine_symptom`, `communication_fault`, or `other`. Returns a dict with `category`, `normalized_complaint`, `severity`, `failure_analysis`, `reasoning`, and `confidence`.

`understand_claim_with_retry()` (lines 380–405): Wraps `understand_claim()` with up to 2 retries and exponential backoff (2^attempt seconds). Returns `None` on failure.

`translate_to_ml_features()` (lines 437–476): Sends a prompt asking the LLM to extract structured features: `customer_complaint` (must match one of 9 known labels), `dtc_codes` (list), `dtc_text` (space-separated string), `dtc_count` (integer), and `has_P/U/C/B` flags.

`format_output()` (lines 273–303): Sends the combined decision data to the LLM and asks it to produce a polished ClaimResponse with proper status, warranty_decision, failure_analysis, reason, matched_complaint, confidence, and decision_engine.

JSON parsing (`_parse_json_response()`) merges the LLM's JSON output with a defaults dict, ensuring all required keys are present even if the LLM omits them. This is critical for robustness — LLMs sometimes drop fields.

Dependencies

- **Depends on:** `openai` (Python client library), `requests` (for OpenRouter HTTP calls), `logging_config`
- **Consumed by:** `ml_predictor.predict()` (for Stages 1, 3, 6)

Design Notes

- `temperature=0` and `seed=42` are set on all LLM calls for deterministic outputs.
- `response_format={"type": "json_object"}` forces the LLM to produce valid JSON, which is then parsed.
- The OpenAI client is initialized lazily via `_get_openai_client()` and cached in a module-level `_openai_client` global.
- The retry logic uses a simple exponential backoff (2^attempt seconds) with a maximum of 2 retries. This is intentionally conservative — LLM calls are slow (~2-5s each) and the system must remain responsive.
- Prompt templates are stored as module-level constants, not in separate files, so they are easy to find and modify.

4.4 `backend/logging_config.py`

Role: Centralized logging configuration with a standard format and a `DecisionLogger` helper class.

Why it exists: Consistent logging across all modules with filename, function, and line number is essential for debugging production issues. The `DecisionLogger` provides structured logging for the multi-stage pipeline.

Key Contents

Element	Type	Purpose
`LOG_LEVEL`	constant	`Reads from LOG_LEVEL env var, defaults to INFO`
`TRACE_FORMAT`	constant	`Log format: %(asctime)s [%(levelname)s] %(name)s %(filename)s:%(funcName)s:%(lineno)d - %(message)s`
`TRACE_DATE_FORMAT`	constant	`"%Y-%m-%dT%H:%M:%S"`
`setup_logging()`	function	`Configures root logger with TRACE format`
`get_logger(name)`	function	Returns a named logger
`DecisionLogger`	class	`Helper for structured decision logging`

How It Works

`DecisionLogger` provides three methods:

- `log_stage(stage, stage_name, **kwargs)` — logs pipeline stage entry with parameters.
- `log_decision(decision_type, result, **context)` — logs a decision result.
- `log_input(func_name, **inputs)` and `log_output(func_name, **outputs)` — debug-level I/O tracing.

Dependencies

- **Standard library only:** `logging`, `os`, `sys`
- **Consumed by:** `ml_predictor.py`, `llm_client.py`, `main.py`, test suite

4.5 `backend/evaluate_model.py`

Role: Comprehensive model evaluation script that computes metrics for both classifiers, runs cross-validation, checks cascade calibration, and evaluates the full Rule+ML pipeline end-to-end.

Why it exists: Model performance must be measured rigorously. The original evaluator had data leakage and cross-validation issues; this version fixes all five known problems (documented in the module docstring) and provides both isolated ML metrics and end-to-end pipeline metrics.

Key Contents

Element	Type	Purpose
`load_data()`	function	`Loads v9 dataset and applies identical preprocessing as train_and_save()`

Element	Type	Purpose
`evaluate_classifier()`	function	`Computes accuracy, precision, recall, F1 (weighted + macro), confusion matrix`
`print_per_class()`	function	`Prints per-class TP, FP, FN, Support`
`check_cascade_calibration()`	function	`Compares FA probability distributions on train vs test data`
`evaluate_pipeline()`	function	`Runs full predict() on held-out data; reports by decision_engine`
`main()`	function	`Orchestrates all evaluations and prints results`

How It Works

1. Loads the model bundle from `trace_models.pkl`.
2. Reproduces the exact same preprocessing pipeline as `train_and_save()`.
3. Splits data using the same `random_state=42` / `test_size=0.2`.
4. Evaluates FA and WD classifiers independently.
5. Runs 3-fold cross-validation on the held-out test set (not training data — this was Fix 2).
6. Checks cascade calibration: compares `clf_fa.predict_proba()` distributions on train vs test.
7. Runs end-to-end pipeline evaluation with full `predict()` calls.
8. Prints feature importance (top 20 for each classifier).

Design Notes

- The `evaluate_pipeline()` function defaults to `sample_size=10` when LLM is enabled (~55s per prediction with LLM) to avoid long runtimes.
- There is a documented concern: `fit_transform` calls in the original evaluator would leak test-set statistics. The current code splits before fitting.
- The cascade calibration check quantifies the train/test distribution shift in FA probabilities — this is a known architectural concern where `clf_wd` was trained on overconfident FA cascade features.

4.6 `backend/requirements.txt`

Role: Python dependency specification.

Package	Version	Purpose
fastapi	`0.111.0`	REST API framework
`uvicorn[standard]`	`0.30.1`	ASGI server
pydantic	`2.7.1`	`Request/response validation`

Package	Version	Purpose
scikit-learn	`1.5.0`	`Preprocessing, metrics, model utilities`
scipy	`1.13.1`	Sparse matrix operations
numpy	`1.26.4`	Array operations
pandas	`2.2.2`	DataFrames
requests	`≥2.31.0`	HTTP client for OpenRouter
python-dotenv	`≥1.0.0`	`.env file loading`
xgboost	`≥2.0.0`	Gradient boosting classifiers
pillow	latest	Image processing for OCR
pytesseract	latest	`OCR (legacy, not currently used)`
easyocr	latest	OCR engine
torch	latest	Backend for EasyOCR
opencv-python-headless	latest	Image preprocessing for EasyOCR
openai	`≥1.0.0`	OpenAI Python client

4.7 `backend/Dockerfile`

Role: Container definition for the backend service.

Builds from `python:3.11-slim`, installs build dependencies, copies source, pre-trains the ML model at build time (`RUN python ml_predictor.py`), and starts uvicorn on port 8000.

Design Note: Pre-training at build time means the container starts instantly — no 10-second model training delay on first request. However, this increases image size significantly due to the dataset and model artifact.

4.8 `frontend/trace.html` (and `frontend/index.html`)

Role: Single-page frontend for submitting warranty claims and displaying results.

Why it exists: Provides a dark-themed, automotive-HUD-inspired UI for submitting DTC codes, voltage readings, and technician notes, then rendering the ClaimResponse with a confidence ring visualization and color-coded verdict badges.

Key Contents

- **Input form:** DTC code text input, voltage number input, technician notes textarea.
- **Analyze button:** Sends `POST /analyze` to the backend.

- **Result panel:** Displays verdict (Approved/Rejected/Needs Manual Review), confidence ring (SVG arc), failure analysis, warranty decision, matched complaint, reason, and engine tag.
- **Confidence ring:** SVG circle where `stroke-dashoffset` is dynamically computed from `confidence` percentage. Color changes: green for Approved, red for Rejected, amber for Needs Manual Review.

`trace.html` is the production version served by nginx. `index.html` is the development version (identical except `API_URL` points to `localhost:8000`).

Dependencies

- No build system; pure vanilla HTML/CSS/JS
- Google Fonts: Rajdhani, Share Tech Mono, Exo 2
- All CSS is inline; no external stylesheets

4.9 `frontend/Dockerfile`

Uses `nginx:1.27-alpine`, copies `trace.html` as `index.html`, and `nginx.conf` as the site config. Listens on port 3000.

4.10 `docker-compose.yml`

Two services:

- **backend:** Builds from `./backend/Dockerfile`, loads `.env`, exposes port 8000, includes health check.
- **frontend:** Builds from `./frontend/Dockerfile`, depends on backend health check, exposes port 3000.

Both connect via `trace_net` bridge network.

4.11 `backend/ml_predictor_DecisionTree.py`

Role: Archived alternative ML model using DecisionTreeClassifier instead of XGBoost.

Why it exists: This was the original v1 implementation trained on a 12,000-row dataset. It uses `DecisionTreeClassifier` with a simpler feature set. It is **not used in production** but is retained for reference and comparison.

4.12 `backend/resave_bundle.py`

Role: Utility script to convert `trace_models.pkl` to `trace_bundle.joblib` with compression level 3.

Why it exists: An experiment in reducing model artifact size. The `.joblib` format with compression can be significantly smaller than uncompressed pickle. Not part of the production pipeline.

4.13 `backend/test_api_timing.py`

Role: Benchmark script that measures the latency of each LLM stage (understand_claim, translate_to_ml_features, format_output) independently.

Why it exists: LLM calls are the primary latency bottleneck. This script isolates each stage's API round-trip time for performance tuning.

4.14 `backend/dataset_gen/`

Role: Series of Python scripts (v3 through v9) that generate the synthetic warranty claims dataset.

Why it exists: Real warranty data is proprietary and unavailable. These scripts produce a synthetic dataset with realistic noise levels (93–96% pattern correlation) to train and evaluate the model. Each version increased dataset size, feature complexity, and noise realism.

---

5. Data Pipeline

5.1 Data Source

The training dataset is `backend/synthetic_warranty_claims_v9.csv` — a 100,000-row synthetic dataset covering vehicle warranty claims from 2019–2025. It is generated by `backend/dataset_gen/generate_dataset_v9(1).py`.

5.2 Loading and Ingestion

In `train_and_save()`:

```
df = pd.read_csv(DATA_PATH)  # DATA_PATH points to synthetic_warranty_claims_v9.csv
```

5.3 Cleaning and Validation

Four fillna operations are applied before modeling:

Column	Fill Strategy	Rationale
`DTC`	`fillna("").replace("none", "")`	`Missing DTC means no codes were stored`
`Customer Complaint`	`fillna("OBD Light ON")`	Default complaint label

Column	Fill Strategy	Rationale
Failure Analysis	fillna("NTF")	No Trouble Found is the default root cause
Warranty Decision	fillna("According to Specification")	Conservative default for missing labels

5.4 Schema / Field Descriptions

Column	Type	Description	Example
DTC	string	Diagnostic Trouble Code(s), comma-separated	"P0562" or "U0100, C0045"
Customer Complaint	string	Canonical complaint label	"Engine overheating"
Failure Analysis	string	Root cause label (6 classes)	"controller failure due to supplier production failure"
Warranty Decision	string	Liability assignment (3 classes)	"Production Failure"
Voltage	float	Measured voltage	14.2
Mileage_km	int	Vehicle mileage in kilometers	52000
Year	int	Vehicle model year	2021
Supplier	string	Supplier identifier	"Bosch"
Date	string	Claim date	"2023-06-15"

5.5 Synthetic Data

The dataset is synthetically generated with **93–96% pattern correlation** (not 100%). This means patterns like "moisture keywords → Customer Failure" hold approximately 91–95% of the time, not always. This mimics real-world data noise and prevents the model from learning trivially perfect rules.

6. Feature Engineering

6.1 Feature Table

#	Feature	Type	Source	Transformation	Why It Was Added
1	Customer Complaint	OHE	Customer Complaint	OneHotEncoder	Strongest signal for warranty decision
2	dtc_text	TF-IDF (40 features)	DTC	TfidfVectorizer	Captures DTC code text patterns
3	dtc_count	numeric	DTC	Count comma-separated codes	CF claims have more codes on average
4	has_P, has_U, has_C, has_B	binary flags	DTC	Binary indicator	DTC prefix is highly predictive
5	dtc_{code} (90+ features)	binary one-hot	DTC	Membership in HIGH_VALUE_DTCS	Specific codes (P0562, U0100, etc.) have strong warranty signal
6	Supplier	OHE	Supplier	OneHotEncoder	Some suppliers have higher failure rates
7	Mileage_km	scaled numeric	Mileage_km	StandardScaler	Higher mileage → more failures
8	Year	scaled numeric	Year	StandardScaler	Newer vehicles → different failure modes
9	mileage_bracket	OHE	Mileage_km	pd.cut → OHE	Non-linear mileage threshold effects on warranty
10	claim_age	scaled numeric	Date, Year	(Date.year - Year)	Direct warranty eligibility signal
11	voltage_bracket	OHE	Voltage	Custom bracketing → OHE	Non-linear voltage thresholds (11V, 15.4V, 17V)
12	dtc_count_bracket	OHE	DTC	Custom bracketing → OHE	CF avg 2.3 codes, PF avg 1.5
13	volt_high_and_P	interaction	Voltage, has_P	(Voltage > 15.4) & (has_P == 1)	High voltage + P-code = strong PF signal
14	volt_low_and_U	interaction	Voltage, has_U	(Voltage < 11.0) & (has_U == 1)	Low voltage + U-code = network fault

#	Feature	Type	Source	Transformation	Why It Was Added
15	`volt_normal_and_C`	interaction	`Voltage, has_C`	`(11 ≤ Voltage ≤ 14.5) & (has_C == 1)`	`Normal voltage + C-code = chassis issue`
16	`has_multiple_prefixes`	interaction	DTC	`(has_P + has_U + has_C + has_B) > 1`	`Multiple DTC categories → more severe`
17	`FA probability vector (6 features)`	cascade	XGBoost FA classifier	`predict_proba output`	`WD classifier sees "what the FA model thinks"`

6.2 Encoding Choices

- **OneHotEncoder** (not OrdinalEncoder) for categorical features because there is no natural ordering between complaint types, suppliers, voltage brackets, etc. `handle_unknown="ignore"` ensures unseen categories at inference time don't crash the model.
- **TF-IDF** with `max_features=40` for DTC text to capture code-text patterns without exploding dimensionality.
- **StandardScaler** for continuous features (Mileage_km, Year, claim_age, Voltage) to normalize scale for XGBoost.
- **LabelEncoder** for target variables (Failure Analysis and Warranty Decision). Fit on full dataset for class coverage, which is safe because these are targets, not features.

6.3 Engineered Features and Their Rationale

- **mileage_bracket:** Warranty decisions are threshold-sensitive to mileage. Brackets ("low", "mid", "high", "very_high") allow OHE to capture non-linear effects that a single scaled numeric would miss.
- **claim_age:** Computed as `Date.year - Year`, this is a direct warranty-eligibility signal. A 1-year-old vehicle filing a claim is very different from a 10-year-old vehicle.
- **voltage_bracket:** Voltage has critical thresholds at 11V (under-voltage), 15.4V (EOS threshold for ASICs), and 17V (extreme over-voltage). Bracketing captures these non-linear jumps.
- **dtc_count_bracket:** CF claims average 2.3 DTC codes vs. 1.5 for PF. Brackets (none, single, few, many) capture this signal.
- **Interaction features:** Cross-terms between voltage and DTC prefix (e.g., high voltage + P-code) are strong indicators of specific failure modes that neither feature alone would capture.

6.4 Features Considered but Rejected

- **Raw voltage as the only feature:** Rejected because voltage has a non-linear relationship with warranty decisions (brackets are better).
- **TF-IDF on technician notes:** Not currently used because the ML model receives pre-categorized complaint labels rather than raw text. LLM Stage 1 handles free-text understanding.
- **Vehicle make/model:** Not available in the dataset.

7. Model Architecture

7.1 Model Selection Rationale

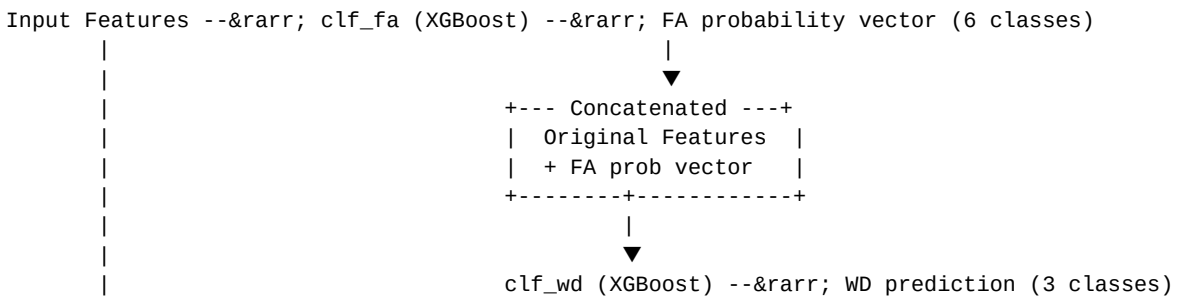
TRACE uses **XGBoost** (`XGBClassifier`) for both classifiers, replacing the original RandomForest. The switch was made based on evaluation findings documented in `docs/xgboost-implementation-plan.md` and `docs/xgboost-tuning-results.md`. Key reasons:

1. **Better accuracy on structured+text features:** XGBoost handles sparse TF-IDF features and one-hot encoded features more effectively.
2. **Regularization built in:** `reg_lambda=0.1` prevents overfitting on the synthetic data.
3. **Calibrated probabilities:** XGBoost tends to produce better-calibrated probabilities than RandomForest, which is critical for the cascade architecture and confidence scoring.

7.2 Model Hyperparameters

Parameter	Value	Rationale
`n_estimators`	1000	`Enough trees for convergence; early stopping not used due to synthetic data`
`max_depth`	10	`Deep enough to capture complex interactions; not excessively deep to avoid overfitting`
`learning_rate`	`0.02`	`Low learning rate for stable convergence with many trees`
`min_child_weight`	3	`Prevents overfitting on small leaf nodes`
`subsample`	`0.8`	Row sampling for robustness
`colsample_bytree`	`0.8`	Column sampling for robustness
`reg_lambda`	`0.1`	L2 regularization
`eval_metric`	mlogloss	Multi-class log loss for monitoring
`random_state`	42	Reproducibility

7.3 Cascade Architecture



```
        |
        +-- FA prob: geometric mean &rarr; ml_confidence
```

The cascade means the WD classifier can "see" what the FA classifier thinks before making its own decision. This creates information flow: if the FA model is confident about "EOS (Electrical OverStress)", the WD model is more likely to predict "Customer Failure".

7.4 Why This Architecture Over Alternatives

- **Why not a single classifier?** The cascade architecture separates the "what went wrong" question (FA) from the "who's responsible" question (WD). These are related but distinct — a single model would conflate them.
- **Why not a neural network?** The feature set is structured (OHE, flags, scaled numerics) with moderate dimensionality (~200+ features). XGBoost excels here and is far more interpretable.
- **Why not end-to-end LLM?** The LLM is used for semantic understanding and text generation, but the core classification must be deterministic, fast, and not dependent on external API availability.

7.5 Rule Engine Within the Architecture

The 9 deterministic rules are not the ML model — they are a separate decision layer that runs before ML. Rules are designed for high-precision, low-recall patterns: when they fire, they are highly confident but they only cover ~25% of claims. The remaining ~75% of claims fall through to the ML classifiers.

---

8. Training Pipeline

8.1 Training Data Preparation

`train_and_save()` in `ml_predictor.py`:

1. Loads `synthetic_warranty_claims_v9.csv` (100K rows).
2. Fills NAs as described in §5.3.
3. Fits `LabelEncoder` on full dataset for FA (6 classes) and WD (3 classes).
4. Applies `extract_dtc_features()` to the DTC column.
5. **Splits 80/20** (`test_size=0.2, random_state=42`) BEFORE any `fit_transform`.
6. Engineers post-split features: `mileage_bracket`, `claim_age`, `voltage_bracket`, `dtc_count_bracket`, interaction features.

8.2 Transformer Fitting (Train-Only)

After the split, 13 transformers are fit on `df_tr` only:

Transformer	Input Column(s)	Output
ohe	Customer Complaint	Sparse OHE matrix

Transformer	Input Column(s)	Output
`tfidf_d`	`dtc_text`	`Sparse TF-IDF matrix (max_features=40)`
`ohe_supplier`	Supplier	Sparse OHE matrix
`mileage_scaler`	`Mileage_km`	Scaled numeric
`year_scaler`	Year	Scaled numeric
`ohe_mileage`	`mileage_bracket`	Sparse OHE matrix
`claim_age_scaler`	`claim_age`	Scaled numeric
`voltage_scaler`	Voltage	Scaled numeric
`ohe_voltage_bracket`	`voltage_bracket`	Sparse OHE matrix
`ohe_dtc_count_bracket`	`dtc_count_bracket`	Sparse OHE matrix

Test data is transformed with the already-fit transformers (`.transform()`, not `.fit_transform()`).

8.3 Cross-Validation Strategy

For the FA cascade probabilities fed into the WD classifier:

```
fa_probs_tr = cross_val_predict(
    XGBClassifier(n_jobs=-1, **_xgb_params),
    X_tr, yfa_tr,
    cv=5,
    method="predict_proba",
)
```

This generates **out-of-fold (OOF)** probability estimates for the training data, preventing the cascade leak where `clf_wd` would otherwise see overconfident in-sample FA probabilities.

8.4 Class Imbalance Handling

No explicit oversampling or undersampling is applied. The synthetic dataset is balanced by design, and XGBoost's `min_child_weight=3` and `subsample=0.8` provide implicit regularization.

8.5 Hyperparameter Choices

See §7.2 for the full list. These were determined through iterative evaluation documented in `docs/xgboost-tuning-results.md`.

8.6 How to Retrain

```
cd backend
python3 -c "from ml_predictor import train_and_save; train_and_save()"
```

Or simply delete `trace_models.pkl` and restart the server — it will auto-train on startup.

9. Inference Pipeline

9.1 End-to-End Prediction Flow

```
predict(fault_code, technician_notes, voltage)
|
+- Stage 1: LLM Understanding
|  +- understand_claim_with_retry(notes, fc)
|     +- Returns: {category, normalized_complaint, severity, failure_analysis, reasoning, confidence}
|     +- Falls back to None if API unavailable
|
+- Stage 2: Rule Engine
|  +- run_rules(fc, notes, voltage)
|     +- Evaluates 9 rules in priority order
|     +- Returns: {rule_fired: bool, rule_id, status, warranty_decision, confidence, ...}
|
+- Stage 3: Feature Preparation
|  +- If LLM available: translate_to_ml_features(notes, fc, category)
|  +- Else: extract_dtc_features(fc) + match_complaint(notes)
|  +- Adds defaults: supplier="Unknown", mileage_km=50000, year=2024, claim_age=1
|
+- Stage 4: XGBoost Scoring
|  +- run_ml(features)
|     +- Builds sparse feature vector
|     +- clf_fa.predict_proba(X) &rarr; FA probability vector
|     +- Augment X with FA probabilities
|     +- clf_wd.predict_proba(X_wd) &rarr; WD probability vector
|     +- confidence = geometric_mean(FA_top_prob, WD_top_prob) x 100, clamped [0, 98]
|
+- Stage 5: Score Combination
|  +- combine_scores(rule_result, ml_result, llm_stage1)
|     +- Weights: rule/ML/LLM based on agreement
|     +- Applies thresholds: &ge;85% firm, 65-85% cautious, &lt;65% manual review
|     +- Tags decision_engine: "LLM+Rule+ML" | "Rule+ML" | "LLM+ML" | "ML"
|
+- Stage 6: Output Formatting
   +- If LLM available: format_output(combined, features)
   +- Else: assemble_output_from_fields(combined, features)
   +- Returns: {status, failure_analysis, warranty_decision, confidence, reason, matched_complaint, decision_
```

9.2 Preprocessing at Inference Time

`run_ml()` builds the feature vector identically to `train_and_save()`, using the saved transformers from the bundle. Key steps:

1. OHE-transform Customer Complaint.
2. TF-IDF-transform DTC text.
3. Extract DTC flag features (has_P, has_U, has_C, has_B, dtc_count, high-value DTC one-hots).
4. OHE-transform Supplier.
5. Scale Mileage_km, Year, claim_age, Voltage.

6. Bracket and OHE-transform mileage_bracket, voltage_bracket, dtc_count_bracket.
7. Compute interaction features (volt_high_and_P, volt_low_and_U, etc.).
8. Stack all via `scipy.sparse.hstack`.

9.3 Rule Engine Priority and Thresholds

Rules are evaluated in order; **first match wins**. Voltage rules come first because they are objective measurements. The unconditional voltage thresholds are:

Rule	Condition	Threshold
`over_voltage`	`voltage > 16.0V`	`16.0`
`low_voltage`	`voltage < 11.0V`	`11.0`

9.4 Confidence Score Calculation

ML confidence:

```
ml_confidence = round(min(98.0, max(0.0, (fa_prob * wd_prob) ** 0.5 * 100)), 1)
```

This is the geometric mean of the top-class probabilities from both classifiers, scaled to 0–100%, capped at 98%.

Combined confidence (Rule + ML agreement, no LLM):

```
combined = 0.70 x rule_conf + 0.30 x ml_conf + 5.0  # agreement bonus
```

With LLM:

```
combined = 0.595 x rule_conf + 0.255 x ml_conf + 0.15 x (llm_conf x 100) + 5.0
```

9.5 Thresholds

Threshold	Value	Effect
`CONFIDENCE_THRESHOLD_FIRM`	`85.0`	`Firm Approved/Rejected decision`
`CONFIDENCE_THRESHOLD_MANUAL`	`65.0`	Below → "Needs Manual Review"
`AGREEMENT_BONUS`	`5.0`	Added when rule and ML agree
`DISAGREEMENT_GAP_THRESHOLD`	`20.0`	Gap between rule and ML confidence
`WEAK_INPUT_PENALTY`	`0.85`	`Multiplier when LLM category is "other" and confidence < 0.3`
`LLM_LOW_CONFIDENCE_THRESHOLD`	`0.3`	`Below this, LLM signal is considered unreliable`

9.6 LLM Fallback Behavior

When no `OPENROUTER_API_KEY` or `OPENAI_API_KEY` is set, or when notes ≤ 5 characters:

- **Stage 1** is skipped (no LLM understanding).
- **Stage 3** uses `extract_dtc_features()` + `match_complaint()` instead of LLM translation.
- **Stage 6** uses `assemble_output_from_fields()` instead of `format_output()`.
- The `decision_engine` tag will be "Rule+ML" or "ML" (no "LLM+" prefix).

9.7 Default Feature Values

When features are not provided (e.g., mileage, supplier, year are not in the API request):

Feature	Default	Rationale
supplier	"Unknown"	Most common placeholder
`mileage_km`	`50000.0`	Mid-range value
year	2024	Recent model year
`claim_age`	1	`Conservative (1 year old vehicle)`
voltage	`14.2 (or actual input)`	Normal charging voltage

---

10. Model Artifacts

10.1 What Gets Saved and Where

File: `backend/trace_models.pkl`

This is a Python pickle `dict` containing:

Key	Type	Description
`clf_fa`	`XGBClassifier`	Failure Analysis classifier
`clf_wd`	`XGBClassifier`	`Warranty Decision classifier (cascade)`
`le_fa`	`LabelEncoder`	`Failure Analysis label encoder (6 classes)`
`le_wd`	`LabelEncoder`	`Warranty Decision label encoder (3 classes)`
`ohe`	`OneHotEncoder`	Customer Complaint encoder
`tfidf_d`	`TfidfVectorizer`	`DTC text encoder (max_features=40)`
`ohe_supplier`	`OneHotEncoder`	Supplier encoder
`mileage_scaler`	`StandardScaler`	`Mileage_km scaler`

Key	Type	Description
year_scaler	StandardScaler	Year scaler
ohe_mileage	OneHotEncoder	Mileage bracket encoder
claim_age_scaler	StandardScaler	Claim age scaler
voltage_scaler	StandardScaler	Voltage scaler
ohe_voltage_bracket	OneHotEncoder	Voltage bracket encoder
ohe_dtc_count_bracket	OneHotEncoder	DTC count bracket encoder

10.2 How to Load and Inspect

```
import pickle
with open("backend/trace_models.pkl", "rb") as f:
    bundle = pickle.load(f)

# Print all keys
print(bundle.keys())

# Inspect a classifier
print(bundle["clf_fa"])
print(bundle["clf_fa"].n_features_in_)

# Inspect label encoders
print("FA classes:", bundle["le_fa"].classes_)
print("WD classes:", bundle["le_wd"].classes_)
```

10.3 Versioning

Model artifacts are versioned through the dataset filename. The current model is trained on `synthetic_warranty_claims_v9.csv`. When a new dataset version is used, `DATA_PATH` in `ml_predictor.py` must be updated and the model must be retrained. There is no automated model versioning or registry — the `.pkl` file is overwritten on each retrain.

---

11. Evaluation & Metrics

11.1 Metrics Tracked

Metric	Formula/Library	Target
Accuracy	sklearn.metrics.accuracy_score	>95%
Precision (weighted)	sklearn.metrics.precision_score(average="weighted")	>90%

Metric	Formula/Library	Target
Recall (weighted)	sklearn.metrics.recall_score(average="weighted")	>90%
F1 (weighted)	sklearn.metrics.f1_score(average="weighted")	>90%
F1 (macro)	sklearn.metrics.f1_score(average="macro")	>85%
Cascade calibration	Mean FA probability gap (train vs test)	<0.05

11.2 Cross-Validation Results

Cross-validation is run as 3-fold on the held-out test set (not training data). The evaluator reports:

```
Failure Analysis CV Accuracy: X.XXXX (+/- Y.YYYY)
Warranty Decision CV Accuracy: X.XXXX (+/- Y.YYYY)
```

Results vary per run; see `evaluate_model.py` output for current numbers.

11.3 Cascade Calibration Check

The evaluator compares `clf_fa.predict_proba()` distributions on training data vs test data:

- If mean gap > 0.05, it warns that `clf_wd` saw overconfident cascade features during training.
- The current code uses `cross_val_predict` for OOF FA probabilities, which mitigates this issue.

11.4 End-to-End Pipeline Evaluation

The evaluator runs the full `predict()` pipeline (Rule Engine → ML → Score Combination) on held-out test rows. This gives realistic accuracy numbers because rules can override ML for ~25% of claims.

Results are broken down by `decision_engine` tag to show whether "Rule+ML", "LLM+Rule+ML", or "ML" decisions are more accurate.

11.5 Feature Importance

The evaluator prints the top 20 most important features for both FA and WD classifiers. Customer Complaint, Supplier, and high-value DTC codes typically dominate.

11.6 Class Labels

Failure Analysis (6 classes):

1. controller failure due to supplier production failure
2. EOS (Electrical OverStress)
3. Sensor short due to moisture

4. NTF
5. Connector damage
6. ASIC CJ327 failure due to EOS

Warranty Decision (3 classes):

1. Production Failure
2. Customer Failure
3. According to Specification

11.7 Known Failure Modes

- **Overlapping rules and ML:** When a rule fires with high confidence but the ML model disagrees, the combine_scores logic may produce counterintuitive statuses. The system always errs toward "Needs Manual Review" in disagreement cases.
- **Default feature values:** When supplier, mileage, year, and claim_age use defaults, the model is making predictions with partial information. This is intentional — the API only requires three inputs.
- **LLM inconsistency:** LLM categorization can vary between calls (despite `temperature=0` and `seed=42`), which may produce different results for identical inputs when LLM is enabled.

---

12. API / Interface Reference

12.1 Endpoints

Method	Path	Description	Request Body	Response
GET	`/`	Health check	None	`{"message": "TRACE Backend Running ■", "version": "2.0"}`
POST	`/analyze`	Analyze warranty claim	`ClaimRequest`	`ClaimResponse`
POST	`/scan-image-easyocr`	`Extract DTC/voltage from image`	`UploadFile`	`{"fault_codes": [...], "voltage": float, "notes": str, "raw_text": str}`

12.2 Request Schema (ClaimRequest)

```
{
  "fault_code": "P0562",
  "technician_notes": "Engine overheating, low idle",
  "voltage": 14.2
}
```

Field	Type	Required	Description
`fault_code`	string	Yes	`DTC code(s), comma-separated`
`technician_notes`	string	Yes	Free-text observations
`voltage`	float	Yes	Measured voltage in volts

12.3 Response Schema (ClaimResponse)

```
{
  "status": "Approved",
  "failure_analysis": "controller failure due to supplier production failure",
  "warranty_decision": "Production Failure",
  "confidence": 85.0,
  "reason": "Rule 'u_code' fired. ML agrees with confidence 85.0%.",
  "matched_complaint": "Engine overheating",
  "decision_engine": "LLM+Rule+ML"
}
```

Field	Type	Values	Description
`status`	string	`Approved, Rejected, Needs Manual Review`	Top-level disposition
`failure_analysis`	string	—	Root cause
`warranty_decision`	string	`Production Failure, Customer Failure, According to Specification`	Liability assignment
`confidence`	float	0–98	`Decision confidence (%)`
`reason`	string	—	Human-readable explanation
`matched_complaint`	string	—	How notes were interpreted
`decision_engine`	string	LLM+Rule+ML, Rule+ML, LLM+ML, ML	Which engines contributed

12.4 Error Handling

Error	Status Code	Detail
Prediction error	500	`Prediction error: <exception message>`
Validation error	422	`Pydantic validation error (missing/invalid fields)`

12.5 CORS Configuration

```
app.add_middleware(
    CORSMiddleware,
```

```
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Note: This allows all origins, which is suitable for development but must be restricted in production.

---

13. Data Flow Walkthrough

13.1 End-to-End Trace: "P0562, Engine overheating, 14.2V"

Input: `fault_code="P0562"`, `technician_notes="Engine overheating, low idle"`, `voltage=14.2`

Stage 1 — LLM Understanding (assuming OPENROUTER_API_KEY is set):

- `understand_claim("Engine overheating, low idle", "P0562")` is called.
- LLM returns: `{category: "engine_symptom", normalized_complaint: "Engine overheating", severity: "high", failure_analysis: "Engine overheating due to cooling system failure", confidence: 0.85}`

Stage 2 — Rule Engine:

- `over_voltage`: 14.2 > 16.0? No.
- `low_voltage`: 14.2 < 11.0? No.
- `moisture`: "engine overheating, low idle" contains moisture keywords? No.
- `physical_damage`: No.
- `ntf`: No.
- `u_code`: "P0562" matches `\bU[0-9A-F]{4}\b`? No.
- `p_code_engine`: "P0562" matches `\bP0[0-9]{3}\b`? Yes. And notes contain "overheat" (keyword)? Yes.
- **Rule fires:** `p_code_engine` → status=Approved, warranty_decision=Production Failure, confidence=80.5%.

Stage 3 — Feature Translation:

- LLM path: `translate_to_ml_features("Engine overheating, low idle", "P0562", "engine_symptom")` returns structured features.
- Fallback would use: `extract_dtc_features("P0562")` → `{dtc_count: 1, has_P: 1, has_U: 0, has_C: 0, has_B: 0, dtc_text: "P0562", dtc_p0562: 1, ...}` and `match_complaint("Engine overheating, low idle")` → "Engine overheating".
- Default values fill in: supplier="Unknown", mileage_km=50000, year=2024, claim_age=1, voltage=14.2.

Stage 4 — XGBoost Scoring:

- Feature vector is built and transformed through all 13 saved transformers.
- `clf_fa.predict_proba(X)` → FA probability vector (6 classes).
- Augmented feature vector: X + FA probability vector.
- `clf_wd.predict_proba(X_wd)` → WD probability vector (3 classes).
- `ml_confidence = (√(FA_top_prob x WD_top_prob)) x 100`, clamped to [0, 98].

Stage 5 — Score Combination:

- Rule fired (p_code_engine, confidence=80.5%) and ML results are combined.
- If ML agrees (warranty_decision = "Production Failure"): `combined = 0.70 x 80.5 + 0.30 x ml_conf + 5.0`.
- If LLM agrees: LLM weight (0.15) is added.
- If combined confidence ≥85%: firm Approved. If 65–85%: Approved with caution. If <65%: Needs Manual Review.

Stage 6 — Output Formatting:

- LLM path: `format_output(combined, features)` produces polished JSON.
- Fallback: `assemble_output_from_fields(combined, features)` produces a template-based response.

Sample Output:

```
{
  "status": "Approved",
  "failure_analysis": "ASIC CJ327 failure due to EOS",
  "warranty_decision": "Production Failure",
  "confidence": 82.3,
  "reason": "Rule 'p_code_engine' fired. ML agrees with confidence 82.3%.",
  "matched_complaint": "Engine overheating",
  "decision_engine": "LLM+Rule+ML"
}
```

13.2 Error Paths

LLM unavailable (no API key):

- Stages 1, 3, and 6 fall back to local code.
- `decision_engine` will be "Rule+ML" or "ML".
- The system produces a valid result, just without LLM enhancement.

LLM call fails transiently:

- `understand_claim_with_retry()` retries up to 2 times with exponential backoff.
- If all retries fail, Stage 1 returns `None` and the pipeline continues without LLM signal.

Model file missing:

- `load_models()` calls `train_and_save()`, which trains from scratch on the dataset.
- This causes a ~10-second cold start delay on the first request.

Invalid input:

- FastAPI validates the request via Pydantic. Missing fields or invalid types return 422.
- Empty `fault_code` or `technician_notes` are handled gracefully by the pipeline (default values, empty DTC features).

13.3 Edge Cases

- **Empty DTC:** `extract_dtc_features("")` returns `{dtc_count: 0, dtc_text: "none", has_P: 0, ...}`. No DTC rules fire; ML still runs.
- **Empty notes:** `match_complaint("")` returns "OBD Light ON" (the most common default). LLM stages are skipped when notes ≤ 5 characters.

- **Extreme voltage (999V):** The over_voltage rule fires (voltage > 16.0). ML still runs, but the combined score will be dominated by the rule's 93% confidence.
- **Multiple DTCs:** "U0100, C0045" — both U-code and C-code rules would match, but only the first match (U-code, priority 6) fires.

---

14. Configuration Reference

14.1 Environment Variables

Variable	Required	Default	Description
`OPENAI_API_KEY`	Conditional	—	API key for OpenAI (gpt-4o-mini). Takes priority over OPENROUTER_API_KEY.
`OPENROUTER_API_KEY`	Conditional	—	API key for OpenRouter. Used when OPENAI_API_KEY is not set.
`LOG_LEVEL`	No	`INFO`	Logging level. Options: DEBUG, INFO, WARNING, ERROR.

Note: At least one API key must be set for LLM features to work. If neither is set, the system still operates using Rule+ML only.

14.2 Module-Level Configuration Constants

These are defined in `ml_predictor.py` and can be adjusted by modifying the source:

Constant	Value	Location	Safe to Change?
`BASE_DIR`	Auto-detected	`ml_predictor.py`	No
`MODEL_PATH`	`BASE_DIR/trace_models.pkl`	`ml_predictor.py`	No (changes require retrain)
`DATA_PATH`	`BASE_DIR/synthetic_warranty_claims_v9.csv`	`ml_predictor.py`	No (changes require retrain)
`CONFIDENCE_THRESHOLD_FIRM`	85.0	`ml_predictor.py`	Yes (affects output status)
`CONFIDENCE_THRESHOLD_MANUAL`	65.0	`ml_predictor.py`	Yes (affects output status)
`AGREEMENT_BONUS`	5.0	`ml_predictor.py`	Yes (affects confidence score)
`DISAGREEMENT_GAP_THRESHOLD`	20.0	`ml_predictor.py`	Yes (affects disagreement logic)

Constant	Value	Location	Safe to Change?
WEAK_INPUT_PENALTY	0.85	ml_predictor.py	Yes (affects low-LLM-confidence scenarios)
RULE_WEIGHT_AGREE	0.70	ml_predictor.py	Yes (affects score combination)
ML_WEIGHT_AGREE	0.30	ml_predictor.py	Yes (affects score combination)
RULE_WEIGHT_DISAGREE	0.55	ml_predictor.py	Yes (affects score combination)
ML_WEIGHT_DISAGREE	0.35	ml_predictor.py	Yes (affects score combination)
LLM_WEIGHT	0.15	ml_predictor.py	Yes (affects LLM signal weight)
LLM_LOW_CONFIDENCE_THRESHOLD	0.3	ml_predictor.py	Yes (affects weak-input penalty)
LLM_LOW_CONFIDENCE_PENALTY	0.7	ml_predictor.py	Yes (affects weak-input penalty)

14.3 LLM Provider Configuration

Parameter	Value	Location
OPENAI_MODEL	"gpt-4o-mini"	llm_client.py
OPENROUTER_MODEL	"arcee-ai/trinity-large-preview:free"	llm_client.py
OPENROUTER_API_URL	"https://openrouter.ai/api/v1/chat/completions"	llm_client.py
temperature	0	All LLM calls
seed	42	All LLM calls
timeout	30 seconds	Default for LLM calls
max_retries	2	understand_claim_with_retry()

14.4 Parameters Requiring Retraining

The following cannot be changed without retraining the model:

- Number and list of `HIGH_VALUE_DTCS` (changes feature dimensions)
- `KNOWN_COMPLAINTS` list (changes OHE vocabulary)
- TF-IDF `max_features` (changes feature dimensions)
- XGBoost hyperparameters (must retrain for changes to take effect)
- Feature engineering logic (mileage_bracket bins, voltage_bracket thresholds)
- Train/test split `random_state` or `test_size`

15. Setup & Running

15.1 Prerequisites

- Python 3.9+ (3.11 recommended)
- pip
- Docker + Docker Compose (for containerized deployment)
- OpenRouter or OpenAI API key (optional — system works without LLM)

15.2 Installation

```
cd backend
pip install -r requirements.txt
```

15.3 Running Locally

```
cd backend

# Set API key (optional)
export OPENROUTER_API_KEY="your-key-here"
# Or: export OPENAI_API_KEY="your-key-here"

# Run the API server
uvicorn main:app --reload --port 8000
```

The first request triggers lazy model loading (~10 seconds). Subsequent requests are fast (~100ms without LLM, ~5-15s with LLM).

15.4 Docker Deployment

```
docker-compose up --build
```

This starts:

- Backend on `http://localhost:8000`
- Frontend on `http://localhost:3000`

The backend Dockerfile pre-trains the model during build, so the container starts instantly.

15.5 Running Tests

```
# All tests (from backend/ directory)
python3 -m pytest

# Specific test files
python3 -m pytest tests/test_ml_predictor.py -v        # Core prediction tests
python3 -m pytest tests/test_e2e.py -v                  # End-to-end tests
python3 -m pytest tests/test_resilience.py -v             # Failure scenario tests

# Without LLM (no API key needed)
```

```
OPENROUTER_API_KEY="" python3 -m pytest tests/test_ml_predictor.py -v
```

15.6 Model Evaluation

```
cd backend
python3 evaluate_model.py
```

This produces a comprehensive report including:

- Isolated FA and WD classifier metrics
- Cross-validation results
- Cascade calibration check
- Feature importance (top 20)
- End-to-end pipeline accuracy

15.7 Common Issues and Fixes

Issue	Cause	Fix
`ModuleNotFoundError: No module named 'xgboost'`	Missing dependency	`pip install xgboost`
`FileNotFoundError: synthetic_warranty_claims_v9.csv`	Wrong working directory	`Run from backend/ directory`
Long cold start on first request	Model training in progress	`Pre-train in Docker: RUN python ml_predictor.py`
LLM calls timing out	Slow API response	`Increase timeout in llm_client.py calls`
`OPENROUTER_API_KEY error`	API key not set	`Create backend/.env with OPENROUTER_API_KEY=your-key`
CORS errors in browser	Frontend on different port	`Backend already allows * origins; check port`

16. Testing

16.1 Test Suite Structure

Test File	What It Covers	Key Test Classes
`test_ml_predictor.py`	Core prediction logic	`TestA1ExtractDTCFeatures, TestA2MatchComplaint, TestA3RunRules, TestA4RunML, TestA5CombineScores, TestA6AssembleOutput, TestA7PredictIntegration, TestB1RulePriority, TestB2CombineLogic, TestB3MLAlwaysRuns, TestB4NoRuleCase, TestB5OutputSchema, TestV9DataPath`
`test_llm_client.py`	LLM client unit tests	API call mocking, JSON parsing
`test_e2e.py`	`End-to-end pipeline with LLM mocking`	`Full pipeline with mocked OpenRouter`
`test_predictor_llm.py`	Predictor + LLM integration	LLM-enabled predictions
`test_openai_integration.py`	OpenAI API integration	Real or mocked OpenAI calls
`test_resilience.py`	Failure scenario resilience	`LLM timeout, invalid responses, missing keys`
`test_logging_config.py`	Logging configuration	Format, level, DecisionLogger
`test_ml_predictor_logging.py`	ML predictor logging	`Decision logging in predict()`
`test_main_logging.py`	API endpoint logging	`Request/response logging`
`test_llm_client_logging.py`	LLM client logging	Stage-specific logging

16.2 What's Covered

- **Unit tests:** Extract DTC features, match complaints, run rules, run ML, combine scores, assemble output.
- **Integration tests:** Full predict() pipeline with and without LLM.
- **Edge cases:** Empty DTC, empty notes, extreme voltage, no rule match, disagreement scenarios.
- **Schema validation:** All 7 required keys present, valid status values, confidence ranges.
- **Resilience:** LLM failure, timeout, invalid JSON, missing API key.

16.3 How to Run Tests

```
cd backend

# All tests
python3 -m pytest

# With verbose output
python3 -m pytest -v

# Specific test class
python3 -m pytest tests/test_ml_predictor.py::TestA3RunRules -v
```

```
# Without LLM (no API key needed)
OPENROUTER_API_KEY="" python3 -m pytest tests/test_ml_predictor.py -v
```

---

17. Known Limitations & Design Decisions

17.1 Synthetic Data Limitations

The training data is **synthetic** with 93–96% pattern correlation. This means:

- The model learns patterns that are *approximately* correct, not perfectly deterministic.
- Accuracy numbers on this dataset are optimistic — real-world performance will be lower.
- Some rule confidences (e.g., `u_code` at 57%) intentionally reflect the noisy nature of the data.

17.2 Default Feature Values

The API only requires three fields (fault_code, technician_notes, voltage), but the ML model needs 13+ features. Default values fill the gap:

- `supplier = "Unknown"` — the OHE has learned this as a category.
- `mileage_km = 50000.0` — mid-range value.
- `year = 2024` — recent year.
- `claim_age = 1` — derived from Date.year - Year with defaults.

These defaults are reasonable but reduce prediction accuracy compared to providing actual data.

17.3 LLM Dependency

The system works without an LLM (Rule+ML mode), but:

- Stage 1 (claim understanding) is skipped — less semantic categorization.
- Stage 3 (feature translation) uses a simpler keyword-matching fallback.
- Stage 6 (output formatting) produces template-based rather than natural-language explanations.
- The overall confidence calibration is less precise without LLM signals.

17.4 Voltage Removal from Frontend

The production frontend (`trace.html`) includes a voltage input field, but some design discussions (see `docs/plans/2026-03-16-voltage-removal.md`) have explored removing it. The backend still processes voltage and the rule engine still evaluates voltage thresholds. **Voltage is not removed from the ML model** — it remains a critical feature for both rules and ML.

17.5 Technical Debt

- **Duplicate functions:** `voltage_bracket()` and `dtc_count_bracket()` are defined identically in both `train_and_save()` and `run_ml()`. Refactoring would require either passing the functions or creating a

shared utilities module.

- **DecisionTree archive:** `ml_predictor_DecisionTree.py` is the original v1 model. It should be clearly marked as archived or moved to `backup/`.
- **Hard-coded paths:** `DATA_PATH` and `MODEL_PATH` use `os.path.dirname(os.path.abspath(__file__))` which works for direct execution but can break in some containerized environments.
- **CORS wildcard:** `allow_origins=["*"]` is insecure for production deployment.

17.6 Trade-offs

Trade-off	Choice	Rationale
Speed vs. Accuracy	XGBoost with 1000 estimators and LLM calls	Accuracy is paramount in warranty decisions; latency is acceptable at ~5-15s per claim
Determinism vs. LLM creativity	Rules override ML, ML overrides LLM	Deterministic rules provide predictable behavior for known patterns
Single model vs. Cascade	Cascade (FA → WD)	The "what went wrong" and " who's responsible" questions are distinct; separate models with information flow
Synthetic vs. Real data	Synthetic 100K rows	Real warranty data is proprietary and unavailable; synthetic data with controlled noise is the practical choice

18. Glossary

Domain Terms

Term	Definition
DTC	Diagnostic Trouble Code — a standardized alphanumeric code (e.g., P0562) stored by a vehicle's ECU indicating a detected fault
ECU	Electronic Control Unit — a computer managing a specific vehicle subsystem (engine, transmission, brakes, etc.)
NTF	No Trouble Found — a claim where no fault could be reproduced during diagnosis
EOS	Electrical OverStress — damage caused by excessive voltage or current
CAN bus	Controller Area Network bus — a communication protocol connecting ECUs in a vehicle

Term	Definition
LIN bus	Local Interconnect Network — a lower-speed vehicle communication bus
OBD	On-Board Diagnostics — the standardized vehicle self-diagnostic system
ASIC	Application-Specific Integrated Circuit — a custom chip (e.g., CJ327) often referenced in failure analysis
Warranty claim	A formal request for warranty coverage of a failed component
Production Failure	A defect caused by the supplier/manufacturer (warranted)
Customer Failure	A defect caused by the vehicle owner (not warranted)
According to Specification	No fault found; vehicle operating within spec (NTF)

ML Terms

Term	Definition
XGBoost	Extreme Gradient Boosting — a high-performance gradient boosting library used for both FA and WD classifiers
Cascade architecture	A model design where one classifier's output (FA probabilities) is fed as features to another (WD)
OOF	Out-of-Fold — cross-validation predictions on training data, used to prevent cascade leakage
TF-IDF	Term Frequency-Inverse Document Frequency — a text vectorization method that weighs terms by their importance
OHE	One-Hot Encoding — representing categorical values as binary indicator vectors
LabelEncoder	scikit-learn utility that maps class labels to integer indices (0, 1, 2, ...)
Feature importance	A measure of how much each feature contributes to model predictions
Cross-validation	A technique for assessing model generalization by training on subsets and testing on held-out subsets
Sparse matrix	A memory-efficient representation of matrices with mostly zero values, used by scikit-learn for text and categorical features

Project-Specific Abbreviations

Abbreviation	Definition
FA	Failure Analysis — the root cause prediction (6-class classification)

Abbreviation	Definition
WD	Warranty Decision — the liability assignment (3-class classification)
Rule Engine	The set of 9 deterministic rules evaluated before ML
Score Combiner	The combine_scores() function that blends rule, ML, and LLM signals
decision_engine tag	A string indicating which layers contributed: "LLM+Rule+ML", "Rule+ML", "LLM+ML", or "ML"
HIGH_VALUE_DTCS	A list of ~90 DTC codes that are one-hot encoded as separate features
KNOWN_COMPLAINTS	A list of 14 canonical complaint labels used for fuzzy matching
Bundle	The pickle file containing all trained models and transformers (trace_models.pkl)

Appendix

A. Full Dependency List with Purpose

Package	Version	Purpose
fastapi	0.111.0	REST API framework with async support and Pydantic validation
uvicorn[standard]	0.30.1	ASGI server for running FastAPI
pydantic	2.7.1	Data validation for request/response schemas
scikit-learn	1.5.0	Preprocessing transformers (OHE, TF-IDF, StandardScaler, LabelEncoder), metrics
scipy	1.13.1	Sparse matrix operations (hstack for combining feature matrices)
numpy	1.26.4	Array operations, argmax, geometric mean
pandas	2.2.2	DataFrame loading, manipulation, and feature engineering
xgboost	≥2.0.0	Gradient boosting classifiers (FA and WD)
requests	≥2.31.0	HTTP client for OpenRouter API calls

Package	Version	Purpose
python-dotenv	≥1.0.0	Loading .env file for API keys
openai	≥1.0.0	OpenAI Python client for gpt-4o-mini
pillow	latest	Image processing for OCR endpoint
easyocr	latest	OCR engine for extracting DTC codes from images
torch	latest	Deep learning framework backend for EasyOCR
opencv-python-headless	latest	Image preprocessing for EasyOCR

B. Dataset Schema Reference

File: `backend/synthetic_warranty_claims_v9.csv` (100,000 rows)

Column	Type	Example	Description
DTC	string	"P0562"	Diagnostic Trouble Code(s), comma-separated
Customer Complaint	string	"Engine overheating"	Canonical complaint label
Failure Analysis	string	"controller failure due to supplier production failure"	Root cause (6 classes)
Warranty Decision	string	"Production Failure"	Liability assignment (3 classes)
Voltage	float	14.2	Measured voltage in volts
Mileage_km	int	52000	Vehicle mileage in kilometers
Year	int	2021	Vehicle model year
Supplier	string	"Bosch"	Supplier identifier
Date	string	"2023-06-15"	Claim date

C. Class Label Definitions

Failure Analysis (6 classes):

Class	Meaning	Typical DTC Pattern
controller failure due to supplier production failure	ECU controller failed due to manufacturing defect	U-series codes
EOS (Electrical OverStress)	Component damaged by excessive voltage	P0-series + high voltage

Class	Meaning	Typical DTC Pattern
Sensor short due to moisture	Sensor or connector damaged by water ingress	Any + moisture keywords
NTF	No Trouble Found — no fault reproduced	Any + NTF keywords
Connector damage	Physical connector or wiring damage	C-series, B-series codes
ASIC CJ327 failure due to EOS	Specific ASIC failure from electrical stress	P0-series + engine symptoms

Warranty Decision (3 classes):

Class	Meaning	Typical Condition
Production Failure	Supplier/manufacturer fault	warranted, approved
Customer Failure	Owner/operator fault	not warranted, rejected
According to Specification	No fault found	NTF, approved

D. Rule Engine Reference Table

#	Rule ID	Condition	Status	Warranty Decision	Confidence
1	over_voltage	voltage > 16.0	Rejected	Customer Failure	93.0%
2	low_voltage	voltage < 11.0	Rejected	Customer Failure	95.0%
3	moisture	keywords: water, moisture, wet, flood, rain, humid, corrosion, corroded	Rejected	Customer Failure	91.0%
4	physical_damage	keywords: crack, broken, impact, collision, bent, misuse, dropped, physical damage	Rejected	Customer Failure	88.5%
5	ntf	keywords: no fault, ntf, no trouble, no issue, no defect, intermittent, cannot reproduce	Approved	According to Specification	95.0%

#	Rule ID	Condition	Status	Warranty Decision	Confidence
6	u_code	DTC matches \bU[0-9A-Fa-f]{4}\b	Approved	Production Failure	57.0%
7	p_code_engine	DTC matches \bP0[0-9]{3}\b AND notes contain: jerk, pickup, acceleration, overheat, fuel, idle, rough	Approved	Production Failure	80.5%
8	c_code	DTC matches \bC[0-9A-Fa-f]{4}\b	Approved	Production Failure	80.0%
9	b_code	DTC matches \bB[0-9A-Fa-f]{4}\b	Approved	Production Failure	80.0%

E. LLM Prompt Templates

E.1 Understanding Claim (Stage 1)

```
You are an automotive warranty analyst. Analyze the claim below and respond ONLY with JSON.

Technician Notes: {notes}
DTC Code: {dtc_code}

Classify into EXACTLY ONE category from this list:
  moisture_damage, physical_damage, ntf, electrical_issue,
  engine_symptom, communication_fault, other

DISAMBIGUATION RULES (apply in order - first match wins):
1. If notes mention overheating, jerking, pickup, acceleration, fuel consumption, idle, rough -&gt; engine_symptom
2. If notes mention CAN bus, LIN bus, communication, network, U-code -&gt; communication_fault
3. If notes mention moisture, water, wet, flood, rain, humidity, corrosion -&gt; moisture_damage
4. If notes mention crack, broken, impact, collision, bent, misuse, dropped, physical damage -&gt; physical_damage
5. If notes mention no fault, ntf, no trouble, no issue, no defect, intermittent, cannot reproduce -&gt; ntf
6. If notes mention electrical short, wiring problems (without engine symptoms) -&gt; electrical_issue
7. Otherwise -&gt; other

Also provide:
- normalized_complaint: one of [9 known labels]
- severity: "low" | "medium" | "high"
- failure_analysis: short root cause string (max 15 words)
- reasoning: brief explanation (max 30 words)
- confidence: float 0.0-1.0

Respond ONLY with JSON: {"category": "...", "normalized_complaint": "...", "severity": "...", "failure_analysis": "
```

E.2 Feature Translation (Stage 3)

```
You are preparing structured features for a machine learning model.
Given the warranty claim below, extract clean structured features.
```

```
Technician Notes: {notes}
DTC Code: {dtc_code}
Pre-classified Category: {llm_category}

Rules:
- customer_complaint MUST be EXACTLY one of: [9 known labels]
- dtc_codes: split comma-separated codes into a list, uppercase, strip spaces
- has_P/U/C/B: 1 if any code starts with that letter, else 0

Respond ONLY with JSON: {"customer_complaint": "...", "dtc_codes": [...], "dtc_text": "...", "dtc_count": 0, "has_P
```

E.3 Output Formatting (Stage 6)

```
You are a warranty claims report writer. Given the structured decision below,
write a clear, professional output for a technician to read.

Decision Data: {combined_json}

Rules:
- status must be EXACTLY: "Approved", "Rejected", or "Needs Manual Review"
- warranty_decision must be EXACTLY one of: "Production Failure", "Customer Failure", "According to Specification"
- failure_analysis: synthesize llm_failure_analysis and ml_failure_analysis into one concise root cause sentence (m
- reason: 1-2 sentences explaining the decision in plain language
- matched_complaint: use customer_complaint from features
- confidence: use combined_confidence exactly as provided (do not change)
- decision_engine: use as provided

Respond ONLY with JSON: {"status": "...", "failure_analysis": "...", "warranty_decision": "...", "confidence": 0.0,
```